

Technical and Editorial Review

Object-Oriented Design Patterns Seminar Report

August 29, 2026

Overall assessment

The manuscript is well organized and substantially developed. The current `main.tex` compiles without LaTeX warnings to a 34-page PDF. The Facade and Strategy chapters contain general and example-specific TikZ diagrams, trade-off tables, application discussions, and seven-question quizzes. The Facade and naive programs compile under C++11 and produce the same device-output sequence. The discussion of `const RouteStrategy*` correctly distinguishes a mutable pointer from access to a const-qualified strategy. The Facade section also correctly distinguishes new client-facing operations from new subsystem functionality.

The report is not yet submission-ready. The Strategy demonstration shows runtime dispatch but does not calculate routes, despite repeated algorithmic claims. The diagrams omit ownership and constness information central to the accompanying explanations. The Decorator chapter remains incomplete against the explicit per-pattern requirements, and its code silently requires C++14. The expanded AI appendix covers more of the preparation history and uses a consistent UTC+07:00 timezone, but it is still a prompt inventory rather than a complete chat log. Final-facing placeholders, an absent required artifact, and citation gaps remain.

No formal derivations, coordinate transformations, inverse-trigonometric expressions, or sign-sensitive equations occur. The required branch, quadrant, sign, and symbol-definition checks were therefore not applicable. The audit did check all numerical examples, implementation signatures, ownership and lifetime claims, diagram relationships, appendices, citations, and references. The required notation and terminology drift check found the concrete UML/type conflicts below.

Technical and evidence findings

1. The Strategy classes are dispatch stubs, not routing algorithms

Classification. Unsupported claim; major.

Location. `main.tex`, lines 755–879 and 1053–1175; the four concrete headers under `Seminar/Strategy/src/strategy/`.

Problem. Every concrete `buildRoute(origin, destination)` ignores both parameters, returns `void`, and prints fixed text, distance, and travel time. `DrivingStrategy`, for example, prints “25 mins (18.5 km)” for every possible location pair. Building the supplied Strategy program with `-Wall`

-Wextra -Wpedantic produces eight unused-parameter warnings. The cited Refactoring.Guru navigation example describes `buildRoute` as using the origin and destination to return route checkpoints, whereas this interface returns no route result.

Why it matters. The code demonstrates interchangeable runtime behavior, but it does not support claims about route calculation, traffic handling, timetable evaluation, elevation optimization, estimates, or tests of real routing algorithms. The literal route figures can appear empirical even though they are independent of input and data.

Suggested fix. Either label the classes explicitly as pedagogical Strategy stubs and remove the fabricated estimates, or implement an input-dependent contract. A minimal credible interface would return a `RouteResult` containing checkpoints or a path, distance, duration, and status. Each strategy should consume the locations or an injected data source. Limit performance and testing claims to behavior the implementation actually provides.

2. The UML diagrams contradict material code semantics

Classification. Definite internal inconsistency; major.

Location. Decorator diagram, lines 223–241; home-theater diagram, lines 479–573; Strategy diagrams and explanation, lines 882–1051.

Problem.

- The Decorator diagram gives `wrapped` the type `Beverage*`; the implementation uses `std::unique_ptr<Beverage>` and claims exclusive ownership.
- The home-theater diagram shows value-like members such as `Amplifier amp`; the implementation stores five references, including `Amplifier& amp`, and relies on non-owning lifetime semantics.
- The navigation diagram shows `RouteStrategy*` and a setter taking that type; the implementation uses `const RouteStrategy*` in both places.
- The actual client constructs all four concrete strategies, but the navigation diagram draws client dependencies only to `DrivingStrategy` and `BicyclingStrategy`.
- The prose says the hollow diamond “expresses” delegation. A hollow diamond denotes shared aggregation; delegation is the behavioral call from `Navigator` to `RouteStrategy`.

Why it matters. Ownership, lifetime, mutability, and delegation are explicit teaching points. The diagrams currently teach different relationships from the code and are unsafe as implementation guides.

Suggested fix. Preserve exact pointer/reference and `const` qualifiers in implementation diagrams. Show `unique_ptr` ownership for Decorator, references or a clearly annotated non-owning association for Facade, and `const RouteStrategy*` for Strategy. Draw all concrete-strategy client dependencies or replace them with one documented generic creation dependency. Use a plain directed association for the non-owning Strategy link unless shared aggregation is intentionally being taught; describe delegation separately.

3. The Decorator listing has an unstated C++14 requirement

Classification. Definite compatibility issue if C++11 is intended; otherwise implementation ambiguity.

Location. `main.tex`, lines 249–283.

Problem. The listing uses `std::make_unique`, which is available from C++14, while the other implementations are explicitly compiled as C++11. Rechecking the displayed code with `g++-std=c++11` fails at both `std::make_unique` calls. The excerpt also omits `<memory>`, `<string>`, and `<utility>` if it is intended to be independently compilable.

Why it matters. Readers receive no indication that one of the three examples requires a different language standard, and the assignment asks for source code rather than an untested fragment.

Suggested fix. State and use C++14 consistently, or provide C++11 construction. Add the required includes, a complete `main`, expected output, and a reproducible compile command. Prefer a standalone Decorator source directory parallel to the Facade and Strategy examples.

4. Reference members do not guarantee dependency validity

Classification. Likely misleading claim.

Location. Facade implementation discussion, lines 618–621.

Problem. The report says references “guarantee non-null dependencies.” References make an explicit null state unavailable during ordinary well-defined construction, but they do not enforce the lifetime of referred objects. Invalid references can also arise through prior undefined behavior. The following sentence correctly acknowledges that all subsystems must outlive the Facade.

Why it matters. The wording sounds like a runtime safety guarantee even though the central remaining risk is a dangling non-owning reference.

Suggested fix. Write: “References make absence unrepresentable in ordinary well-defined construction and communicate non-ownership; the subsystem objects must outlive the Facade.” Keep null-state and lifetime claims separate.

5. Demonstration headers rely on transitive includes

Classification. Likely portability issue.

Location. `Seminar/Facade/src/facade/Projector.h`, line 10, its naive counterpart, and `Seminar/Facade/src/facade/HomeTheaterFacade.h`, line 22.

Problem. These headers name `std::string` without directly including `<string>`. They compile only because other included headers happen to provide the declaration under the current include order.

Why it matters. A header should compile when included first in a minimal translation unit; otherwise unrelated include-order or library changes can break the program.

Suggested fix. Add `#include <string>` to every header that directly names `std::string`, then syntax-check each project header in isolation.

6. Quantitative values are not consistently defined or derived

Classification. Implementation ambiguity; minor but concrete.

Location. Decorator prices at lines 258–282; Strategy outputs at lines 1071–1091 and in the concrete source headers.

Problem. Decorator values 2.00 and 0.40 have no stated currency or unit. Strategy times and distances have units, but they are fixed literals rather than calculations. The Facade light level is consistently specified as a percentage and presents no dimensional conflict.

Why it matters. An undefined price unit impedes interpretation, while fixed route estimates may be mistaken for computed or measured results.

Suggested fix. Define the beverage currency and mark all values as sample data. Use integer minor units or a decimal money type if the price example is more than illustrative. Remove route estimates until they are derived from inputs, or label them explicitly as canned output.

Coverage and submission findings

7. Decorator fails several explicit per-pattern requirements

Classification. Definite completeness problem; major.

Location. Seminar/Requirements.md, items 1–10 under “For each pattern”; the complete Decorator chapter, lines 199–294.

Problem. Decorator has no 5–8 question multiple-choice quiz, no separate general and beverage-specific class diagrams, and no developed explanation of web and mobile applications. Its naive designs are prose-only despite the stated objective of comparing naive and pattern-based C++ designs. No standalone compilable Decorator source accompanies the source directories for the other two patterns.

Why it matters. These elements are explicitly required for each pattern. The detailed Facade and Strategy chapters make the Decorator chapter’s incompleteness and evidence imbalance especially visible.

Suggested fix. Add a naive C++ implementation, separate general and beverage diagrams, tested pattern source, explicit advantages and disadvantages, explained web/mobile examples, and a 5–8 item ABCD quiz with an answer key.

8. The expanded prompt inventory is still not a complete AI Chat Log

Classification. Definite internal inconsistency; possible submission compliance issue.

Location. AI Usage Notes, lines 1339–1385; AI Chat Log, lines 1396–1577.

Problem. The appendix now covers preparation prompts from July through August and consistently converts exposed times to UTC+07:00. However, it contains user prompts only, not assistant responses or resulting actions, so it remains a prompt inventory rather than a chat log. One item still says “Timestamp: not exposed.” The declared access period ends at 16:33 on August 29, while the last listed prompt is at 11:18 that day; the statement that available individual access times are

recorded in the following log does not reconcile with those entries. The disclosure itself says that the complete Prism history still needs to be exported or captured separately.

Why it matters. The assignment explicitly requests an AI Chat Log. A partial prompt list cannot show the AI responses, generated content, or what the group accepted, and may be mistaken for the required complete record.

Suggested fix. Rename the appendix “Prompt Summary” unless it is expanded to the complete conversation. Reconcile the access-period endpoint with the final logged event, explain the unavailable timestamp, and submit the actual exported two-sided conversation as the authoritative log. Confirm the required format with the instructor.

9. Final-facing placeholders and artifact claims remain unresolved

Classification. Definite completeness and consistency problem.

Location. Group-member tables, lines 107–140; Submission Checklist, lines 1317–1330; Member Contribution Record, lines 1579–1584.

Problem. Both responsibility fields still contain bracketed example instructions. The earlier tables already give names and 50% shares, but the later appendix still says names and percentages “should be entered.” The checklist states that remaining artifacts are maintained separately, yet `Changes.md`, explicitly required by the assignment, is absent from the workspace at review time.

Why it matters. These contradictions are visible evidence of an unfinished submission and weaken the checklist’s reliability.

Suggested fix. Replace the responsibility placeholders with verified contributions, update or remove the stale contribution instruction, create and review `Changes.md`, and turn the checklist into an evidence-based status list rather than a general assertion.

10. Citation provenance does not match the revised content

Classification. Definite citation-integrity problem, with some claims requiring external verification.

Location. Pattern definitions and applications; AI Chat Log, especially the Facade comparison prompts; references, lines 1586–1609.

Problem. The body cites only `gof` and `refactoringguru`. The entries `headfirst`, `sourcecmaking`, and `cpp` are unused. The `refactoringguru` entry points specifically to the Strategy page, although the recorded work also used the separate Facade page to add the Additional Facade participant and revise terminology; that material has no matching body citation or bibliography entry. The Decorator intent and broad application list are also uncited.

Why it matters. A reference supports a claim only when the correct source is cited at the relevant location. The current list does not separate formal definitions, implementation facts, and illustrative analogies.

Suggested fix. Add and cite a separate Facade source, cite the source of the Decorator intent, and cite version-specific C++ claims if the ISO entry is retained. Support or qualify application examples such as Python decorators, API Gateways, ORMs, and mobile APIs. Remove genuinely unused entries and add access dates if required by the course style.

Meaningful editorial issues

AI-log naming. “AI Chat Log” overstates a section containing only selected prompts. “Prompt Summary” is accurate unless the full two-sided transcript is inserted. This affects meaning and compliance, not merely style.

Algorithm terminology. Until route computation exists, replace “routing algorithm” and “calculates routes” around the demonstration with “strategy stub,” “mode-specific demonstration behavior,” or similarly bounded wording.

Participant naming. Use role names such as “Concrete Component” in prose and class-like identifiers such as `ConcreteComponent` only in UML or code. Consistent typography would better distinguish abstract roles from beverage classes.

Proofing sweep

The bundled mechanical scan was run against the compiled 34-page manuscript. Its 80 double-period hits were table-of-contents leader dots, not punctuation defects. Source-level spot checks found no genuine duplicated comma, semicolon-capitalization anomaly, malformed “into ... into” frame, JavaScript/iOS/iPhone capitalization error, or `arctan(x/y)` branch ambiguity. The previous duplicated word in the bicycling source is no longer present. The bibliography contains no obvious duplicated punctuation or broken quotation punctuation. Two high-confidence final-facing defects remain:

- **Location:** member tables, lines 120–133. **Problem:** bracketed “[Example: ...]” instructions remain. **Fix:** replace them with final verified responsibility statements.
- **Location:** Member Contribution Record, line 1583. **Problem:** “Member names and percentages should be entered” is stale because the earlier tables already contain names and 50% values. **Fix:** replace it with the final contribution summary or a reference to the completed official form.

Highest-priority fixes

1. Make Strategy claims match the code: implement input-dependent route results or explicitly present the classes as dispatch stubs.
2. Complete Decorator so every per-pattern assignment requirement is met and provide reproducibly compilable source.
3. Align all UML types and relationship semantics with implementation ownership, lifetime, and constness.
4. Replace the prompt-only appendix with, or clearly distinguish it from, the required complete AI Chat Log.
5. Resolve contribution placeholders, the stale appendix statement, and the missing `Changes.md` artifact.
6. Repair citation provenance and the remaining C++ header/standard issues.

Items requiring external verification

Confirm with the instructor whether the AI Chat Log must contain a full two-sided transcript, screenshots, an export file, or only prompts, and whether timestamps and exact model/platform labels are mandatory. Verify the required reference style, URL policy, and access-date format. Finally, verify broad real-world examples—especially Python function decorators, API Gateways, ORMs, camera APIs, and middleware—against authoritative documentation or pattern literature. They may be useful analogies, but the manuscript does not currently establish that each is a canonical Gang of Four instance.